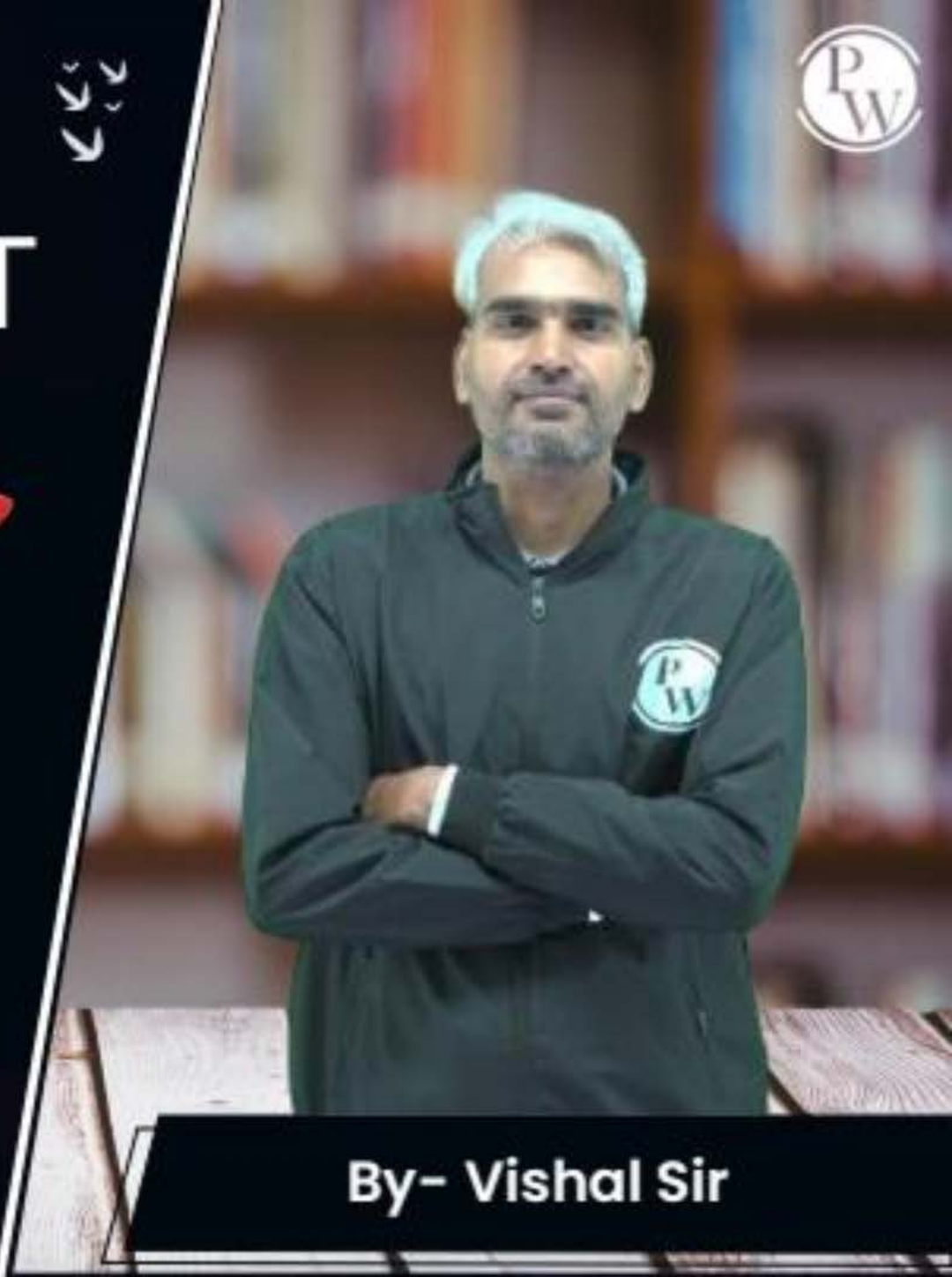
Computer Science & IT



Database Management System

Query Languages

Lecture No. 02



By- Vishal Sir

Recap of Previous Lecture



- **Topic** Query Languages
- **Topic** Relational algebra
- **Topic** Basic relational algebra operations

Slide

Topics to be Covered



- Topic: Derived relational algebra operations
- Topic: Different types of join operations
- Topic: Division operation

Slide



Topic : Rename (ρ)

Consider the following relational schema

Students(Stud_id, Stud_name)

Rename operation can be used to rename attribute of the relation, name of the relation or both.

- Renaming a relation:

$$\rho_{FinalYrStudents}(Students)$$

Current name of relation

new_name for relation

- Renaming attributes:

$$\rho_{(SID,\ Sname)}(Students)$$

name of relation

New name for 1st attribute

New name for 2nd attribute

- Renaming both:

$$\rho_{FinalYrStudents\ (SID,\ Sname)}(Students)$$

New name for Rel^{n} & attributes



Topic : Rename (ρ)

Consider the following relational schema

Students(Stud_id, Stud_name)

Rename operation can be used to rename attribute of the relation, name of the relation or both.

- Renaming a relation:

$\rho_{FinalYrStudents}(Students)$

o/p = FinalYrStudents(Stud_id, Stud_name)

- Renaming attributes:

$\rho_{(SID,\ Sname)}(Students)$

o/p = Students(SID, Sname)

- Renaming both:

$\rho_{FinalYrStudents\ (SID,\ Sname)}(Students)$

FinalYrStudents(SID, Sname)



Topic : Derived Relational Algebra operators

1. Intersection (∩)
2. Join Operations ("⋈")
3. Division Operation (÷)

* It is a set opⁿ, it can be performed only if relation are union compatible

⇓

Intersection is a derived relational algebra opⁿ, it is derived using "Set difference"

$R \cap S = R - (R - S)$



Topic : Join Operations

- Join operations are used to join relational tables based on some condition.

  * If we do not specify any condition, then it becomes Cross Join / Cross product.
  * Using cross join, all the tuples of the first relation will be joined with all the tuples of 2nd relation.



Topic : Types Join Operations

1) Inner join
   a. Theta join
   b. Equi join
   c. Natural join

{ Using inner join, we select only those tuples from both the relations that satisfy the join condition }

2) Outer join
   a. Left outer join
   b. Right outer join
   c. Full outer join

{ In outer join we can also select the tuples from one or both the relations, that failed the join Condⁿ }



Topic : Inner join

- Inner join, includes only those tuples that satisfy the join condition.



Topic : Theta join

The general case of JOIN operation is called a Theta join. It is denoted by symbol θ

i.e. $\bowtie_\theta$ — it may be any Cond^n.

- Theta join can use any conditions in the selection criteria.

$<$, $>$, $\leq$, $\geq$, $=$, $<>$ (Not equal)

✓ Student (S)

Sid	Sname
2	A
3	B
4	A

✓ Employee (E)

Eid	Ename
2	B
4	A
5	B

We can use any Condⁿ in theta join

Student ⋈ Employee

(Student.Sid < Employee.Eid)

it is the join Condⁿ

≡

$$\left(\sigma_{S.Sid < E.Eid}(S \times E)\right)$$

O/P :

No projection oph is used

∴ All attributes of "S×E" will be present in output.

S.Sid	S.Sname	E.Eid	E.Ename
2	A	4	A
2	A	5	B
3	B	4	A
3	B	5	B
4	A	5	B



Topic : Equijoin

{ Equi-join is a type of theta join where Cond^{n} is equality Condition }

An equijoin is a theta join using the equality operator.
In equijoin the condition is always equality condition.

- It is a type of theta join.

└ But it need not be on Common attributes

Student (S)

Sid	Sname
2	A
3	B
4	A

Employee (E)

Eid	Ename
2	B
4	A
5	B

Attribute need not be Common attribute

$$\text{Student} \bowtie_{(\text{Student.Sid} = \text{Employee.Eid})} \text{Employee}$$

Equality cond^n :. it can be Called Equi-join

$$\equiv \left(\sigma_{S.Sid = E.Eid} (S \times E) \right)$$

o/p :

S.Sid	S.Sname	E.Eid	E.Ename
2	A	2	B
4	A	4	A

Student (S)

Sid	Sname
2	A
3	B
4	A

Enroll (E)

Sid	Cid
2	C_1
4	C_2
5	C_3

Attribut may be Common attribute as well

Student $\bowtie$ (Student.Sid = Enroll.Sid) Enroll

Equi-join

$\equiv \left(\sigma_{S.Sid = E.Sid}(S \times E)\right)$

It is not natural join. In natural join we do not specify join condn.

O/P :

S.Sid	S.Sname	E.Sid	E.Cid
2	A	2	C_1
4	A	4	C_2

When we use Equi-join, Common attributes from both the relation will be Present in o/p

Note:- If we use theta join or Equi-join then we need to specify the join condition Explicitly. Which is not the case with natural join.

In natural join we do not specify any Condⁿ; because by default Condⁿ for natural join is Equality Condition on all Common attributes.



Topic : Natural join (⋈)

Natural join is performed based on the equality condition on all common attributes (column) between the relations. The name and type of common attributes must be same.
Additionally, a natural join removes the duplicate columns involved in the equality comparison so only one of each compared column remains

Student (Sid, Sname) & Enroll (Stu-id, Cid)
There is no common attribute between Student & Enroll w.r.t. Natural join

Natural Join ($\bowtie$)

Natural join is a derived relational algebra oph. which can be derived using three basic relational algebra oph. i.e; Cross product, Selection and Projection.

Derivation of Natural join

$R \bowtie S =$

Step-1 : obtain R x S

Step-2 : Select the tuples from R x S based on Equality Condn on all common attributes of R and S.

Step-3 : From the output of Step-2 project distinct attributes

Student (S)

Sid	Sname
2	A
3	B
4	A

Enroll (E)

Sid	Cid
2	C_1
4	C_2
5	C_3

$$\text{Student} \bowtie \text{Enroll} = \pi_{Sid, Sname, Cid}\left(\sigma_{Student.Sid = Enroll.Sid}\left(\text{Student} \times \text{Enroll}\right)\right)$$

No Condn is specified
↓
∴ Natural join

O/P =

"Sid" will be present only once in the o/p

Sid	Sname	Cid
2	A	C_1
4	A	C_2

eg: $R(A\ B\ C)$ & $S(B\ C\ D)$

$$R \bowtie S = \pi_{A,B,C,D}\left(\sigma_{\substack{R.B = S.B \\ \wedge \\ R.C = S.C}} (R \times S) \right)$$

Equality Condⁿ ✓ on all common attributes

eg. $R(\bar{A}, \bar{B}, \bar{C})$ & $S(\bar{D}, \bar{E})$ ✓

$R \bowtie S = \pi_{A,B,C,D,E}(\underbrace{\quad\quad}_{\text{No Common attributes b/w R \& S}}(R \times S)) \equiv R \times S$

∴ No selection Condⁿ.

↓

And hence all the tuples of R×S will get selected

And finally all attributes of R×S will be projected.

∴ O/p will be exactly same as R×S

If there is no common attribute between R and S, then Natural join of R and S will be equal to "R×S"

Student (S)

Sid	Sname
2	A
3	B
4	A

Employee (E)

Eid	Ename
2	B
4	A
5	B

$$\underset{(S)}{\text{Student}} \bowtie \underset{(E)}{\text{Employee}} \equiv S \times E$$

No Common attributes b/w Student & Enroll



Topic : Outer join

In an outer join, along with tuples that satisfy the join condition, we also include tuples that do not match the criteria either from left hand side relation, or from right hand side relation or both.



Topic : Left Outer Join (⟕)

$R \text{ ⟕ } S$ = All tuples of $R \bowtie S$, along with the tuples from left hand side relation {i.e. Rel^n R} that failed the join Cond^n are present.



Topic : Right Outer Join (⟖)

$R \rightarrow\!\!\!\bowtie S$: All tuples of $R \bowtie S$ along with the tuples of right hand side relation {i.e, Rel^n S} that failed the join condition are present



Topic : Full Outer Join (⟗)

R ⟗ S : All tuples of R ⋈ S along with the tuples from both side relation that failed the join condition are present

R =

	A	B	C
✓	1	2	3
✗	1	4	5

S =

	B	C	D
✓	2	3	4
✗	5	6	7
✗	4	7	9

R ⋈ S =

A	B	C	D
1	2	3	4

R ⟕ S =

A	B	C	D
1	2	3	4
1	4	5	NULL

R ⟖ S =

A	B	C	D
1	2	3	4
NULL	5	6	7
NULL	4	7	9

R ⟗ S =

A	B	C	D
1	2	3	4
1	4	5	NULL
NULL	5	6	7
NULL	4	7	9



Topic : Division (÷)

- Division operation is used whenever the query is with respect to every or all.

Eg. Consider following three relations.

Student (S)

Sid	Sname
S_1	A
S_2	A
S_3	B
S_4	C

Enroll (E)

Sid	Cid	fee
S_1	C_1	-
S_1	C_2	-
S_2	C_2	-
S_1	C_3	-
S_2	C_3	-
S_3	C_3	-

Course (C)

Cid	Cname
C_1	OS
C_2	COA
C_3	DBMS

Query:- Retrieve Sids of the students who have enrolled for all Courses.

Enroll (E)

Sid	Cid
S_1	C_1
S_1	C_2
S_2	C_2
S_1	C_3
S_2	C_3
S_3	C_3

$\Pi_{Sid}(E) \Rightarrow$ O/P =

Sid
S_1
S_2
S_3

This R.A. Expression Will produce the Sids of the Students Who have enroll for Some (at least) Course

Course (C)

Cid	Cname
C_1	OS
C_2	COA
C_3	DBMS

$\pi_{Cid}\ (C) \Longrightarrow$ O/P =

Cid
C_1
C_2
C_3

This R.A. Expression will produce Cids of all Courses.

* $\pi_{sid}(E)$ =

Sid
S_1
S_2
S_3

✓

* $\pi_{cid}(C)$ =

Cid
C_1
C_2
C_3

✓

$\underbrace{\pi_{sid}(E) \times \pi_{cid}(C)}$ => o/p =

This R.A. Expression will produce a universal relation, in which Every student who has Enrolled for some Courses is Combined with all Courses.

E.Sid	C.Cid
S_1	C_1
S_1	C_2
S_1	C_3
S_2	C_1
S_2	C_2
S_2	C_3
S_3	C_1
S_3	C_2
S_3	C_3

Student (S)

Sid	Sname
S_1	A
S_2	A
S_3	B
S_4	C

Enroll (E)

Sid	Cid	fee
S_1	C_1	
S_1	C_2	
S_2	C_2	
S_1	C_3	
S_2	C_3	
S_3	C_3	

Course (C)

Cid	Cname
C_1	OS
C_2	COA
C_3	DBMS

* Retrieve Sids of the students who have Enrolled for all Courses.

↳ We are actually looking for the Sids in the Enroll table which are associated with all the Cids of the Course table.

desired result can be produced by following relational algebra Expression

$$\pi_{Sid,Cid}(Enroll) \div \pi_{Cid}(Course)$$

it will produce Sids from Enroll table Which are associated with all Cids of Course table

Student (S)

Sid	Sname
S_1	A
S_2	A
S_3	B
S_4	C

Enroll (E)

Sid	Cid	fee
S_1	C_1	
S_1	C_2	
S_2	C_2	
S_1	C_3	
S_2	C_3	
S_3	C_3	

Course (C)

Cid	Cname
C_1	OS
C_2	COA
C_3	DBMS

* Derivation of division ($\div$) operation using basic relational algebra operation

$$\underbrace{\pi_{sid,cid}(E) \div \pi_{cid}(C)}$$

$$\equiv \left[\pi_{sid}(E) - \pi_{sid}\left[\left(\pi_{sid}(E) \times \pi_{cid}(C)\right) - \pi_{sid,cid}(E)\right]\right]$$

$$\pi_{sid,cid}(E) \div \pi_{cid}(C)$$

$$\equiv \pi_{sid}(E) - \pi_{sid}\left[\pi_{sid}(E) \times \pi_{cid}(C) - \pi_{sid,cid}(E)\right]$$

① ⑧ ⑦ ② ④ ③ ⑥ ⑤





2 mins Summary

Topic Derived relational algebra operations

Topic Different types of join operations

Topic Division operation

THANK - YOU